ARQUITETURA ORIENTADA A EVENTOS EM NUVEM: UMA PROPOSTA BASEADA EM SERVIÇOS AWS PARA APLICAÇÕES ESCALÁVEIS

EVENT-DRIVEN ARCHITECTURE IN THE CLOUD: AN AWS-BASED PROPOSAL FOR SCALABLE APPLICATIONS

Pedro Vitor Ribeiro Silva[1]
CPF: [omitido na versão pública]
SENAI "Gaspar Ricardo Júnior" – Sorocaba/SP
pedrovitorrbeirosilva@gmail.com

Orientador: Mestre Deivison Shindi Takatu[2]
ORCID: 0000-0002-8210-1002
CPF: [omitido na versão pública]
SENAI "Gaspar Ricardo Júnior" – Sorocaba/SP
deivison.takatu@sp.senai.br
Co-orientador: Doutor Glauco Todesco[3]
ORCID: 0000-0003-0603-0641
CPF: [omitido na versão pública]
SENAI "Gaspar Ricardo Júnior" – Sorocaba/SP
glauco.todesco@sp.senai.br

RESUMO: Aplicações *web* modernas precisam absorver picos de carga, evoluir continuamente e manter alta disponibilidade, requisitos que arquiteturas monolíticas atendem com dificuldade. Este trabalho, desenvolvido como projeto de Iniciação Científica, propõe uma Arquitetura Orientada a Eventos (*Event-Driven Architecture* – EDA) construída sobre serviços gerenciados da Amazon Web Services (AWS). A pesquisa é aplicada, qualitativa e exploratória: a partir de um levantamento bibliográfico e da comparação com trabalhos relacionados, definiram-se requisitos arquiteturais, registraram-se as alternativas consideradas para cada um e projetou-se uma solução em cinco camadas – apresentação, entrada, mensageria, processamento e persistência/observabilidade – que integra CloudFront, API Gateway, Cognito, SNS, SQS, Lambda, DynamoDB, RDS, S3 e CloudWatch. A análise qualitativa, fundamentada na literatura, indica que a mensageria assíncrona no modelo *publish/subscribe*, combinada a funções *serverless*, tende a reduzir o acoplamento, isolar falhas e permitir escalabilidade horizontal com custo proporcional ao uso; discutem-se também mecanismos para mensagens duplicadas, eventos fora de ordem e consistência entre bases. Como limitação, esses ganhos são esperados, e não demonstrados: a proposta ainda não foi validada experimentalmente, e a implementação de um protótipo e a medição de latência e custo constituem os próximos passos.

Palavras-chave: Arquitetura orientada a eventos; Computação em nuvem; Escalabilidade; Microsserviços; *Serverless*.

ABSTRACT: Modern web applications must absorb load peaks, evolve continuously and remain highly available, requirements that monolithic architectures struggle to

[1] Graduando em Tecnologia em Análise e Desenvolvimento de Sistemas no SENAI "Gaspar Ricardo Júnior", Sorocaba-SP. E-mail: pedrovitorrbeirosilva@gmail.com
[2] Docente do SENAI "Gaspar Ricardo Júnior". Mestre em Ciência da Computação (UFSCar). E-mail: deivison.takatu@sp.senai.br
[3] Docente do SENAI "Gaspar Ricardo Júnior". Doutor em Engenharia Elétrica (USP). E-mail: glauco.todesco@sp.senai.br

meet. This work, developed as an undergraduate research project, proposes an Event-Driven Architecture (EDA) built on managed services from Amazon Web Services (AWS). The research is applied, qualitative and exploratory: based on a literature survey and a comparison with related work, architectural requirements were defined, the alternatives considered for each one were recorded and a five-layer solution was designed – presentation, entry, messaging, processing and persistence/observability – integrating CloudFront, API Gateway, Cognito, SNS, SQS, Lambda, DynamoDB, RDS, S3 and CloudWatch. The qualitative, literature-based analysis indicates that asynchronous publish/subscribe messaging combined with serverless functions tends to reduce coupling, isolate failures and enable horizontal scalability with pay-per-use costs; mechanisms for duplicate messages, out-of-order events and consistency across data stores are also discussed. As a limitation, these gains are expected rather than demonstrated: the proposal has not yet been experimentally validated, and implementing a prototype and measuring latency and cost are the next steps.

Keywords: Cloud computing; Event-driven architecture; Microservices; Scalability; Serverless.

1 INTRODUÇÃO

A transformação digital ampliou a dependência das organizações em relação a sistemas de *software* distribuídos, que precisam responder em tempo real a volumes crescentes de usuários e dados. A popularização da computação em nuvem, com elasticidade, alta disponibilidade e cobrança sob demanda, viabilizou novos estilos arquiteturais (Rapôso; Costa Junior; Ferreira, 2024), e a migração de aplicações monolíticas para microsserviços tornou-se recorrente na indústria (Deprá *et al.*, 2021). Nesse cenário, a Arquitetura Orientada a Eventos (*Event-Driven Architecture* – EDA), na qual os componentes se comunicam por eventos publicados em um intermediário de mensagens, destaca-se por favorecer o desacoplamento e a resiliência (Villaça; Azevedo, 2018). Este trabalho apresenta uma proposta de EDA sobre serviços gerenciados da Amazon Web Services (AWS).

1.1 PROBLEMA DE PESQUISA

Sistemas monolíticos exigem a reimplantação completa da aplicação a cada alteração, concentram pontos únicos de falha e só escalam como um bloco, o que gera gargalos e custos desproporcionais sob picos de demanda (Garcia, 2023). A questão que orienta a pesquisa é: como estruturar, em nuvem pública, uma arquitetura

de *software* escalável, resiliente e desacoplada, capaz de processar eventos de forma assíncrona sem impor à equipe de desenvolvimento um esforço operacional elevado?

1.2 JUSTIFICATIVA

Para demonstrar a lacuna, seis trabalhos nacionais que propõem, aplicam ou analisam arquiteturas de microsserviços foram comparados segundo seis critérios (Quadro 1): uso de eventos e de intermediário de mensagens; uso de serviços gerenciados ou *serverless* em nuvem pública; tratamento explícito de segurança; observabilidade; tratamento de mensagens duplicadas, ordem e consistência; e avaliação experimental. Villaça e Azevedo (2018) propõem microsserviços orientados a eventos, com plataforma de *streaming*, para integrar bases poliglotas, mas não tratam nuvem, segurança nem observabilidade, e a avaliação é apenas planejada. Prestes *et al.* (2021) e Stradolini (2022) adotam o RabbitMQ em contêineres: o primeiro, em *data center* universitário e sem métricas de desempenho; o segundo migra um módulo de pagamentos para Kubernetes, usa um banco gerenciado do Google Cloud e discute segurança e monitoramento, mas descarta o modelo *serverless* e avalia o resultado de forma qualitativa. Costa e Antonelli (2025) adotam orquestração síncrona em Azure, com mensageria apenas opcional, e Dantas (2021) e Lima (2019) tratam eventos somente no plano conceitual. Nenhum dos trabalhos analisados combina EDA com serviços *serverless* gerenciados, nem discute de forma explícita mensagens duplicadas, eventos fora de ordem e consistência entre bases. O diferencial desta proposta é reunir esses elementos em uma arquitetura de referência única sobre a AWS, com a relação entre requisitos, alternativas e escolhas documentada. Como nenhum dos trabalhos – inclusive este – apresenta medições quantitativas de desempenho, a avaliação experimental permanece uma lacuna comum, tratada aqui como trabalho futuro. Uma referência aberta e documentada auxilia equipes pequenas e médias a tomar decisões fundamentadas e serve de material de estudo para estudantes de graduação.

Quadro 1 – Comparação com trabalhos relacionados

Trabalho	Eventos e *broker*	Nuvem gerenciada ou *serverless*	Segurança	Observa-bilidade	Duplicatas, ordem e consistência	Avaliação experimental
Villaça e Azevedo (2018)	S	N	N	N	P	N
Lima (2019)	P	N	P	P	N	N
Dantas (2021)	P	N	N	P	N	N
Prestes *et al.* (2021)	S	N	N	N	N	P
Stradolini (2022)	S	P	S	S	P	P
Costa e Antonelli (2025)	P	P	S	P	N	P
Esta proposta	**S**	**S**	**S**	**S**	**S**	**N**

Fonte: Elaborado pelo autor (2026). Legenda: S = sim; P = parcial (tratado apenas conceitualmente, como opção ou sem medição quantitativa); N = não.

1.3 OBJETIVOS

O objetivo geral é propor e descrever uma arquitetura orientada a eventos, baseada em serviços gerenciados da AWS, que atenda a requisitos de escalabilidade, desempenho e resiliência. Os objetivos específicos são: (i) identificar, na literatura, padrões arquiteturais aplicáveis a sistemas escaláveis; (ii) descrever os componentes da solução e o papel de cada um; (iii) discutir os mecanismos de segurança e escalabilidade adotados; e (iv) consolidar documentação técnica reaproveitável. Como hipótese de trabalho, espera-se – com base na literatura consultada (Porto, 2009; Garcia, 2023; Villaça; Azevedo, 2018) – que mensageria assíncrona e funções *serverless* proporcionem ganhos de escalabilidade em relação a abordagens monolíticas e síncronas, sem complexidade operacional incompatível com equipes de médio porte. Esses ganhos são, portanto, uma expectativa derivada da literatura, e não um resultado demonstrado por esta pesquisa, que não realizou medições.

2 METODOLOGIA

Trata-se de pesquisa de natureza aplicada, abordagem qualitativa e caráter exploratório, conduzida em quatro etapas. (1) Levantamento bibliográfico: foram reunidos artigos de periódicos e de eventos da SBC, dissertação, trabalhos de conclusão de curso e literatura técnica sobre arquitetura de *software*, microsserviços, computação em nuvem e escalabilidade; os trabalhos que propõem ou avaliam arquiteturas foram comparados entre si (Quadro 1). (2) Definição de requisitos: a partir do problema, estabeleceram-se cinco requisitos – R1, suportar picos de carga sem

indisponibilidade; R2, permitir a evolução e a implantação independentes dos componentes; R3, reduzir o acoplamento e isolar falhas entre módulos; R4, diminuir o esforço operacional por meio de serviços gerenciados; e R5, garantir segurança e rastreabilidade das operações. (3) Projeto arquitetural: para cada requisito, levantaram-se as alternativas tecnológicas disponíveis e registrou-se a justificativa da escolha (Quadro 2); a solução foi documentada em descrição textual e diagrama. (4) Análise qualitativa: as decisões de projeto foram confrontadas com os padrões de integração, consistência, segurança e escalabilidade descritos na literatura.

3 REFERENCIAL TEÓRICO

A computação em nuvem é um modelo de provisionamento de recursos computacionais sob demanda, acessíveis pela rede, com elasticidade e cobrança por uso (Mell; Grance, 2011). Rapôso, Costa Junior e Ferreira (2024) apontam que sua adoção promove mudanças estruturais nas arquiteturas de *software*, tornando escalabilidade horizontal, modularidade e resiliência atributos centrais dos sistemas.

A arquitetura de microsserviços decompõe a aplicação em serviços pequenos e autônomos, com implantação independente. Em levantamento com 23 empresas de *software* de Maringá-PR, Deprá *et al.* (2021) verificaram que escalabilidade, manutenção e evolução foram os principais motivos de migração para esse estilo, citados por 77,8% das empresas que o adotam. A revisão multivocal de Morais, Cordeiro e OliveiraJr (2025) mapeia os tópicos, desafios e tecnologias associados ao tema. Os ganhos, porém, têm contrapartida: ao comparar os modelos monolítico e de microsserviços no setor financeiro, Costa e Antonelli (2025) observam que modularidade e manutenção são beneficiadas, mas o desacoplamento exige maior atenção à infraestrutura e à latência.

Na EDA, a comunicação ocorre pela produção e pelo consumo de eventos, com três papéis: o produtor, o intermediário (*broker*) e o consumidor. O desacoplamento temporal e espacial promovido pelo *broker* permite que os serviços evoluam de forma independente, processem mensagens em paralelo e reprocessem eventos em caso de erro; padrões como canal *publish/subscribe*, fila de mensagens inválidas e receptor idempotente estão catalogados por Hohpe e Woolf (2003). Villaça e Azevedo (2018)

aplicam esse estilo a microsserviços para integrar dados frequentemente atualizados oriundos de bases poliglotas.

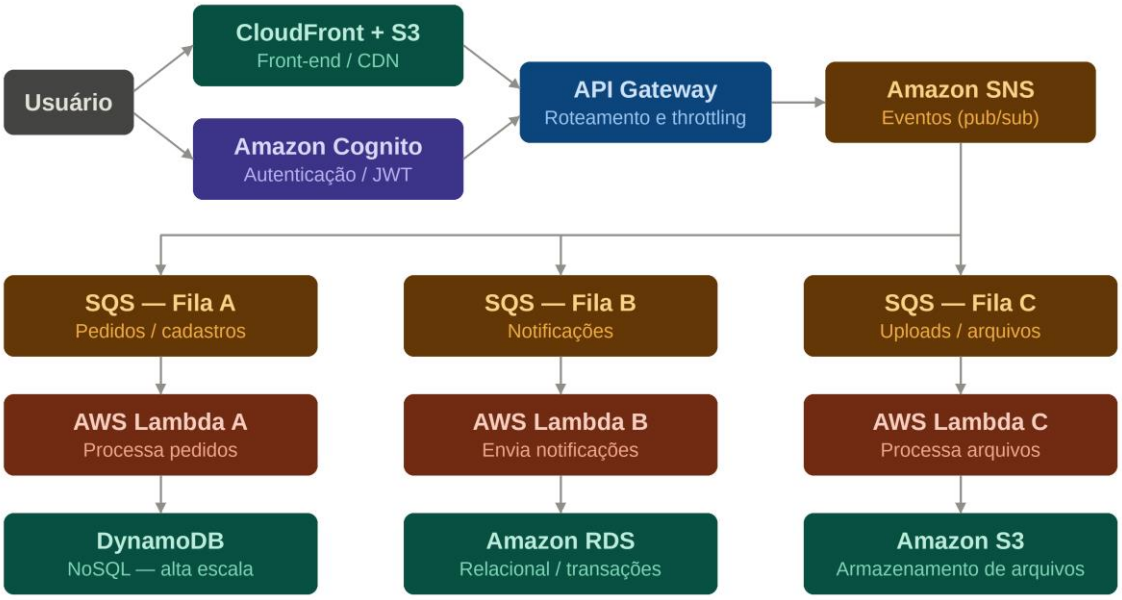
Quanto à escalabilidade, Porto (2009) sistematiza padrões e diretrizes arquiteturais para o escalonamento horizontal, e Garcia (2023) destaca o uso de filas, particionamento, replicação e *cache* em sistemas de grande escala. Esses princípios são compatíveis com funções *serverless*, que não mantêm estado entre invocações e cuja capacidade é provida e cobrada pelo provedor conforme o uso (Jonas *et al.*, 2019).

4 RESULTADOS E DISCUSSÃO

O principal resultado é a arquitetura apresentada na Figura 1, projetada integralmente sobre serviços gerenciados e organizada em cinco camadas: apresentação, entrada, mensageria, processamento e persistência/observabilidade.

Na apresentação, a interface em React/Next.js é hospedada no Amazon S3 e distribuída pelo CloudFront, com o AWS WAF na borda. Na entrada, o API Gateway realiza roteamento e *throttling* e valida os *tokens* JWT emitidos pelo Amazon Cognito. Requisições simples são atendidas de forma síncrona por funções AWS Lambda; operações de maior latência tornam-se eventos de domínio (por exemplo, “pedido criado” ou “arquivo enviado”) publicados em tópicos do Amazon SNS. Filas do Amazon SQS assinam esses tópicos e entregam as mensagens a funções Lambda especializadas, cada uma responsável por um único tipo de evento, que persistem dados no DynamoDB (operacionais, de alta vazão) ou no RDS (transacionais), gravam arquivos no S3 por meio de URLs pré-assinadas e enviam notificações pelo Amazon SES. CloudWatch, X-Ray e CloudTrail concentram métricas, *logs*, rastreamento e auditoria.

Figura 1 – Arquitetura orientada a eventos proposta



Fonte: Elaborado pelo autor (2026).

A organização em camadas decorre diretamente dos requisitos (Quadro 2). A apresentação atende a R1 e R5, pois entrega conteúdo estático em *cache* na borda, sem servidores de aplicação, e filtra tráfego malicioso antes que ele alcance a API. A entrada atende a R4 e R5, centralizando autenticação, limitação de taxa e roteamento sem código próprio. A mensageria é o núcleo de R2 e R3: o produtor publica o evento sem conhecer seus consumidores, e cada fila isola o ritmo e as falhas do seu consumidor. O processamento atende a R1 e R4, escalando por evento sem servidores a administrar. A persistência/observabilidade atende a R1 e R5, separando os dados por perfil de acesso e registrando métricas, rastros e auditoria. A contribuição específica dessa organização não está nos serviços isolados, já documentados pelo provedor, mas na sua composição: (i) a separação explícita entre o caminho síncrono, reservado a requisições simples, e o assíncrono, destinado às operações de maior latência; (ii) a regra de um consumidor por tipo de evento, que transforma a inclusão de uma funcionalidade em uma nova assinatura, sem alterar o produtor; e (iii) a rastreabilidade entre requisitos, alternativas e decisões, que torna a proposta replicável por equipes pequenas e utilizável no ensino.

Quadro 2 – Requisitos, alternativas consideradas e justificativas das escolhas

Requisito	Escolha	Alternativas consideradas	Justificativa
R1 – Picos de carga	Lambda; DynamoDB sob demanda; filas SQS	EC2 com Auto Scaling; contêineres em ECS/Fargate; DynamoDB provisionado	Escala por requisição, sem capacidade mínima a dimensionar; as filas absorvem rajadas (Garcia, 2023).
R2 – Evolução independente	Um consumidor Lambda por tipo de evento	Monólito modular; microsserviços com chamadas REST síncronas	Novos consumidores entram sem alterar o produtor; chamadas síncronas propagam mudanças de contrato.
R3 – Desacoplamento e isolamento de falhas	SNS (pub/sub) + SQS com DLQ	EventBridge; Amazon MQ; Kinesis Data Streams; Amazon MSK (Kafka)	Combina distribuição (fan-out) e fila durável com reprocessamento nativo; Kinesis e MSK priorizam fluxos contínuos e exigem dimensionar partições; MQ mantém brokers em instâncias dedicadas.
R4 – Menor esforço operacional	API Gateway, Cognito e Lambda gerenciados	Servidores em EC2; cluster Kubernetes (EKS); provedor de identidade autogerido	Elimina a gestão de sistema operacional, atualizações e cluster; cobrança por uso (Jonas *et al.*, 2019).
R5 – Segurança e rastreabilidade	WAF, Cognito (JWT), IAM, KMS; CloudWatch, X-Ray e CloudTrail	Autenticação implementada na aplicação; ferramentas de terceiros (ELK, Prometheus/Grafana, Jaeger)	Integração nativa entre serviços e trilha de auditoria sem infraestrutura adicional, ao custo de maior dependência do provedor.

Fonte: Elaborado pelo autor (2026).

Quanto à escalabilidade, o Lambda escala por execuções concorrentes, o DynamoDB ajusta a capacidade no modo sob demanda e as filas SQS funcionam como amortecedor, evitando que produtores rápidos sobrecarreguem consumidores lentos – aplicação direta dos padrões de filas e de componentes sem estado descritos por Porto (2009) e Garcia (2023). Como a cobrança ocorre por uso, o custo acompanha a demanda. Quanto à resiliência, a falha de um consumidor não se propaga aos demais: as mensagens permanecem na fila, são reprocessadas e, após tentativas sem sucesso, seguem para *Dead Letter Queues* para análise posterior. A segurança é construída em camadas: WAF no perímetro, autenticação federada no Cognito, políticas de menor privilégio no AWS IAM e criptografia em trânsito (TLS) e em repouso (AWS KMS).

A discussão também evidencia contrapartidas. A consistência passa a ser eventual e a entrega “ao menos uma vez” exige consumidores idempotentes; o rastreamento de um fluxo distribuído depende da camada de observabilidade, em linha com o alerta de Costa e Antonelli (2025) sobre infraestrutura e latência; e a opção por um único provedor implica dependência tecnológica (*vendor lock-in*).

Três situações exigem tratamento explícito. Mensagens duplicadas, decorrentes da entrega “ao menos uma vez” do SNS e do SQS padrão, são tratadas pelo padrão de consumidor idempotente (Hohpe; Woolf, 2003; Richardson, 2018): cada evento carrega um identificador único gerado pelo produtor, e o consumidor o

registra em uma tabela do DynamoDB por escrita condicional, com tempo de expiração, antes de executar o efeito colateral, descartando as repetições. Eventos fora de ordem são tratados de duas formas: nos fluxos em que a sequência importa, como as mudanças de estado de um mesmo pedido, usam-se tópicos SNS FIFO e filas SQS FIFO com o identificador da entidade como grupo de mensagens, o que garante a ordem por entidade sem serializar todo o sistema, ao custo de menor vazão; nos demais, cada evento leva um número de versão, e a atualização só é aplicada se for mais recente que a versão armazenada, de modo que um evento atrasado não sobrescreve um estado novo (Kleppmann, 2017).

Para evitar inconsistências entre o DynamoDB e o RDS, descarta-se a transação distribuída. A operação é gravada primeiro no banco de origem, e a publicação do evento decorre dessa gravação – pelo DynamoDB Streams ou, no RDS, por uma tabela de saída (padrão *transactional outbox*) –, o que impede que o dado seja salvo sem que o evento seja emitido, ou o contrário. Processos de múltiplas etapas seguem o padrão *saga*, com ações compensatórias em caso de falha (Richardson, 2018), e uma rotina periódica de reconciliação compara os registros dos dois bancos. Por fim, todos esses mecanismos – e os benefícios apontados nesta seção – decorrem de análise qualitativa, e não de medições: ainda não foram implementados e serão verificados no protótipo, de modo que a hipótese permanece sustentada teoricamente, mas não verificada de forma empírica.

5 CONSIDERAÇÕES FINAIS

Os objetivos propostos foram atendidos no plano conceitual: propôs-se e documentou-se uma arquitetura orientada a eventos sobre serviços gerenciados da AWS, com a descrição dos componentes, do fluxo de dados, da relação entre requisitos e escolhas e dos mecanismos de segurança, escalabilidade e consistência previstos. Os ganhos de escalabilidade e a redução da complexidade operacional, contudo, permanecem como hipótese: são coerentes com a literatura consultada, mas esta pesquisa ainda não dispõe de evidências experimentais que os comprovem. A contribuição do trabalho é, portanto, uma proposta arquitetural fundamentada e rastreável, e não a demonstração do seu desempenho.

As principais limitações são a ausência de validação experimental – que impede afirmar ganhos de desempenho, custo ou esforço operacional – e o foco em um único provedor de nuvem. Como trabalhos futuros, pretende-se: (i) implementar um protótipo funcional, incluindo os mecanismos de idempotência, ordenação e consistência descritos; (ii) comparar custo, latência e vazão com uma arquitetura monolítica equivalente, sob carga controlada; (iii) avaliar a portabilidade para outros provedores; e (iv) incorporar práticas de FinOps para otimização contínua de custos.

AGRADECIMENTOS

Ao SENAI “Gaspar Ricardo Júnior” e aos professores Deivison Shindi Takatu e Glauco Todesco, pela orientação e pelo acompanhamento ao longo do projeto de Iniciação Científica.

REFERÊNCIAS BIBLIOGRÁFICAS

COSTA, R. F.; ANTONELLI, H. L. Modernização de arquitetura de sistemas: uma abordagem para transformação digital no sistema financeiro. *In*: ESCOLA REGIONAL DE INFORMÁTICA DO ESPÍRITO SANTO (ERI-ES), 10., 2025. **Anais** [...]. Porto Alegre: SBC, 2025. p. 140-149. DOI: 10.5753/eries.2025.15930. Disponível em: https://sol.sbc.org.br/index.php/eries/article/view/38408. Acesso em: 10 jun. 2026.

DANTAS, F. C. **Uma proposta de arquitetura de software limpa baseada em microsserviços**. 2021. Trabalho de Conclusão de Curso (Graduação) – Universidade Federal Rural do Semi-Árido, Mossoró, 2021. Disponível em: https://repositorio.ufersa.edu.br/handle/prefix/6466. Acesso em: 10 nov. 2026.

DEPRÁ, T. G. B. *et al*. Estudo exploratório sobre o uso da arquitetura de microsserviços na indústria de software. **Revista Eletrônica de Iniciação Científica em Computação**, Porto Alegre, v. 19, n. 3, 2021. DOI: 10.5753/reic.2021.1999. Disponível em: https://journals-sol.sbc.org.br/index.php/reic/article/view/1999. Acesso em: 10 jun. 2026.

GARCIA, V. C. **Padrões de arquitetura para escalabilidade**: uma perspectiva prática. Recife, 2023. Disponível em: https://viniciusgarcia.me/architecture/padroes-de-rquitetura-para-escalabilidade/. Acesso em: 10 jun. 2026.

HOHPE, G.; WOOLF, B. **Enterprise integration patterns**: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

JONAS, E. *et al*. **Cloud programming simplified**: a Berkeley view on serverless computing. Berkeley: University of California, 2019. (Technical Report UCB/EECS-2019-3). Disponível em: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-3.html. Acesso em: 10 nov. 2026.

KLEPPMANN, M. **Designing data-intensive applications**: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

LIMA, M. M. **Análise do processo de migração de sistemas de arquitetura monolítica para microsserviço**. 2019. Trabalho de Graduação (Engenharia da Computação) – Centro de Informática, Universidade Federal de Pernambuco, Recife, 2019. Disponível em: https://www.cin.ufpe.br/~tg/2019-1/TG_EC/tg_mml3.pdf. Acesso em: 10 nov. 2026.

MELL, P.; GRANCE, T. **The NIST definition of cloud computing**. Gaithersburg: National Institute of Standards and Technology, 2011. (NIST Special Publication 800-145). DOI: 10.6028/NIST.SP.800-145.

MORAIS, L. Z.; CORDEIRO, A. F. R.; OLIVEIRAJR, E. Arquitetura de microsserviços: uma revisão multivocal. *In*: ESCOLA REGIONAL DE ENGENHARIA DE SOFTWARE (ERES), 9., 2025, Chapecó. **Anais** [...]. Porto Alegre: SBC, 2025. p. 21-30. DOI: 10.5753/eres.2025.16759. Disponível em: https://sol.sbc.org.br/index.php/eres/article/view/40375. Acesso em: 10 jun. 2026.

PORTO, I. O. **Padrões e diretrizes arquiteturais para escalabilidade de sistemas**. 2009. Dissertação (Mestrado em Ciência da Computação) – Universidade Federal de Uberlândia, Uberlândia, 2009. Disponível em: https://repositorio.ufu.br/bitstream/123456789/12488/1/ivens.pdf. Acesso em: 10 jun. 2026.

PRESTES, A. N. *et al*. Uma arquitetura de microsserviços para análise de dados de mobilidade urbana baseada em dados heterogêneos. *In*: SIMPÓSIO BRASILEIRO DE REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS (SBRC), 39., 2021, Uberlândia. **Anais Estendidos** [...]. Porto Alegre: SBC, 2021. p. 249-256. DOI: 10.5753/sbrc_estendido.2021.17178. Disponível em: https://sol.sbc.org.br/index.php/sbrc_estendido/article/view/17178. Acesso em: 10 nov. 2026.

RAPÔSO, C. F. L.; COSTA JUNIOR, O. G.; FERREIRA, A. S. Impactos da computação em nuvem na arquitetura de software: uma análise de literatura. **Revista Tópicos**, [*s. l.*], 2024. DOI: 10.5281/zenodo.13949416. Disponível em: https://revistatopicos.com.br/artigos/impactos-da-computacao-em-nuvem-na-arquitetura-de-software-uma-analise-de-literatura. Acesso em: 10 jun. 2026.

RICHARDSON, C. **Microservices patterns**: with examples in Java. Shelter Island: Manning, 2018.

STRADOLINI, C. J. **Migração de sistemas monolíticos para microsserviços**: estudo de caso de migração de um módulo de pagamentos de e-commerce. 2022. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Instituto de Informática, Universidade Federal do Rio Grande do Sul, Porto Alegre, 2022. Disponível em: https://lume.ufrgs.br/handle/10183/243216. Acesso em: 10 nov. 2026.

VILLAÇA, L. H. N.; AZEVEDO, L. G. Uma arquitetura de microsserviços orientada a eventos para integração de dados frequentemente atualizados oriundos de bases de dados poliglotas. *In*: SIMPÓSIO BRASILEIRO DE SISTEMAS DE INFORMAÇÃO (SBSI), 14., 2018, Caxias do Sul. **Anais Estendidos** [...]. Porto Alegre: SBC, 2018. p. 100-104. Disponível em: https://sol.sbc.org.br/index.php/sbsi_estendido/article/view/6205. Acesso em: 10 jun. 2026.